11. Design a Notification Service

Key Terms:

Term	Full Name	What it does
APNs	Apple Push Notification service	Apple's push delivery to iOS devices
FCM	Firebase Cloud Messaging	Google's push delivery to Android/iOS
SES	Simple Email Service	AWS email sending service
DLQ	Dead Letter Queue	Queue for messages that failed processing
2FA	Two-Factor Authentication	Security requiring two verification methods

Problem Statement

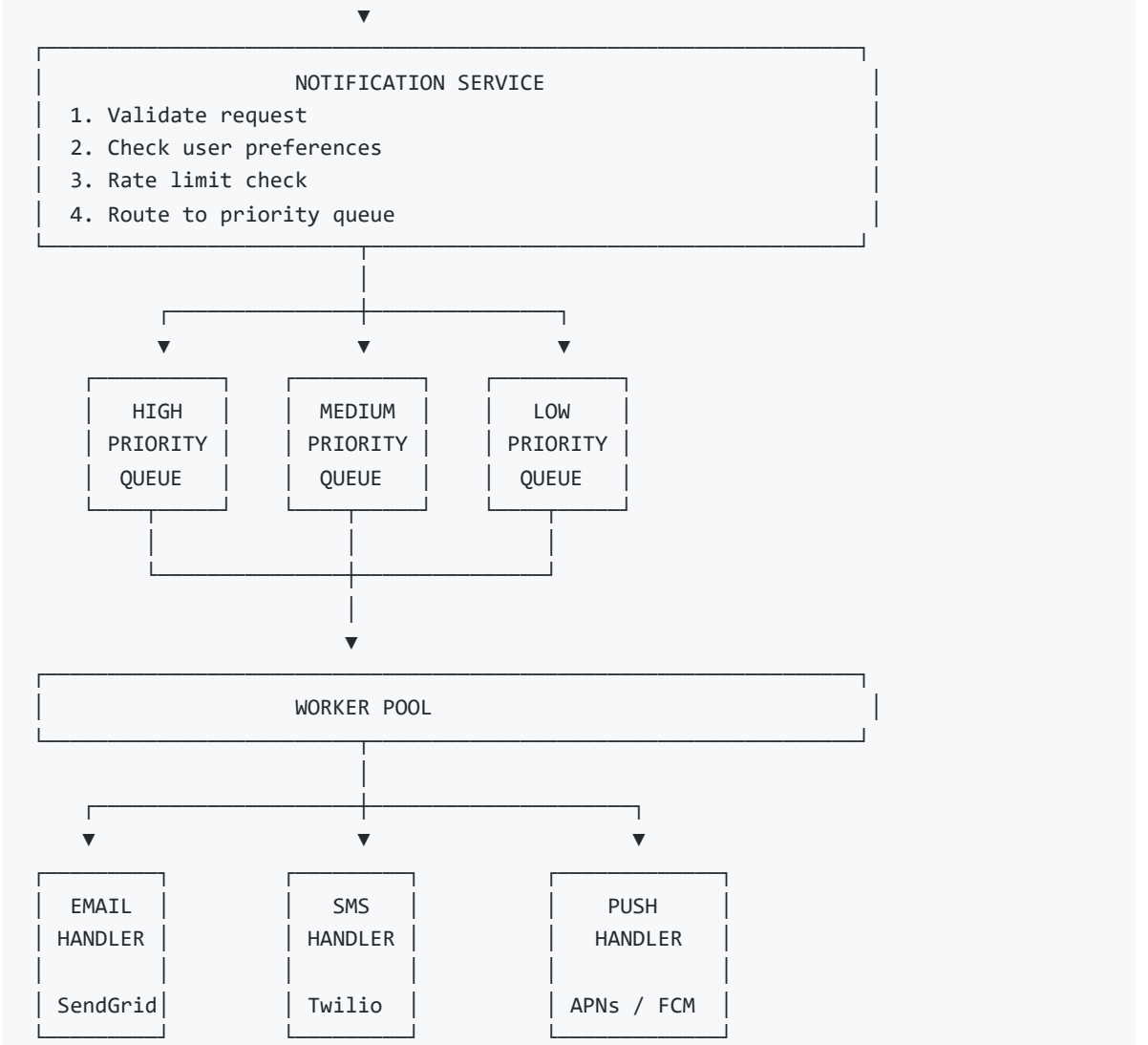
Design a scalable notification system that delivers email, SMS, and push notifications to millions of users with high reliability and low latency.

Clarifying Questions to Ask

1. **Channels**: Email, SMS, push, in-app, or all?
2. **Volume**: How many notifications per day? (1B?)
3. **Latency**: Real-time or batched acceptable?
4. **Priority**: Support urgent vs. normal notifications?
5. **Preferences**: User opt-in/out per channel?
6. **Templates**: Dynamic content or fixed templates?
7. **Tracking**: Track delivery, opens, clicks?

High-Level Design





API Design

Notification APIs

Method	Endpoint	Description
`POST`	`/api/v1/notifications/send`	Send a notification
`POST`	`/api/v1/notifications/batch`	Send bulk notifications
`GET`	`/api/v1/notifications?userId={id}&limit=20`	Get user's notification history
`PUT`	`/api/v1/notifications/{notifId}/read`	Mark notification as read
`PUT`	`/api/v1/notifications/read-all`	Mark all as read

Preference APIs

Method	Endpoint	Description
`GET`	`/api/v1/users/{userId}/preferences`	Get notification preferences
`PUT`	`/api/v1/users/{userId}/preferences`	Update notification preferences
`POST`	`/api/v1/users/{userId}/devices`	Register device for push notifications
`DELETE`	`/api/v1/users/{userId}/devices/{deviceId}`	Unregister device

Send Notification Request

```
POST /api/v1/notifications/send
{
  "user_id": "user_123",
  "type": "order_shipped",
  "channels": ["push", "email", "in_app"],
  "priority": "high",
  "template_id": "tmpl_ship_01",
  "data": {
    "order_id": "ORD-456",
    "tracking_url": "https://..."
  }
}
```

Key Components

Component	Purpose	Technology
API Gateway	Receive notification requests	REST API
Preference Service	User notification settings	PostgreSQL
Template Service	Render dynamic content	Handlebars/Mustache
Queue	Buffer and prioritize	Kafka / RabbitMQ
Workers	Process and send	Distributed workers
Email Provider	Send emails	SendGrid, SES
SMS Provider	Send SMS	Twilio, Nexmo
Push Provider	Mobile push	APNs, FCM

Notification Flow

```
1. SERVICE SENDS REQUEST:
   POST /notifications
   {
     "user_id": "123",
     "template_id": "order_shipped",
     "channels": ["email", "push"],
     "data": {"order_id": "456", "tracking": "ABC123"},
     "priority": "high"
   }

2. VALIDATION & ROUTING:
   - Check user exists and preferences
   - Check rate limits (max 10 emails/hour per user)
   - Enqueue to appropriate priority queue

3. WORKER PROCESSING:
   - Dequeue message
   - Render template with data
   - Call provider API
   - Handle response (success/failure)
   - Store delivery status

4. RETRY LOGIC:
   - Exponential backoff: 1s, 2s, 4s, 8s...
   - Max 5 retries
   - Dead letter queue for failures
```

Database Schema

```sql
-- Notifications
CREATE TABLE notifications (
    id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    template_id VARCHAR(50),
    channel VARCHAR(20),  -- email, sms, push
    status VARCHAR(20),   -- pending, sent, delivered, failed
    priority INT,
    data JSONB,
    created_at TIMESTAMP,
    sent_at TIMESTAMP,
    delivered_at TIMESTAMP,
    error_message TEXT
);

-- User Preferences
CREATE TABLE notification_preferences (
    user_id BIGINT,
    channel VARCHAR(20),
```

```
    enabled BOOLEAN DEFAULT true,
    quiet_hours_start TIME,
    quiet_hours_end TIME,
    PRIMARY KEY (user_id, channel)
);
```

Scalability Considerations

- **Priority queues**: Process high-priority (2FA codes) before newsletters
- **Rate limiting**: Per-user and per-channel limits
- **Batching**: Batch digest emails (daily summary instead of per-event)
- **Multi-provider**: Fallback to secondary provider if primary fails
- **Async processing**: Never block the caller

Trade-offs

Decision	Option A	Option B	Recommendation
Queue	Single queue	Priority queues	**Priority** - urgent first
Delivery	At-least-once	Exactly-once	**At-least-once** - simpler
Templates	Inline	Centralized	**Centralized** - reusable
Tracking	Basic (sent)	Full (opens)	**Full** - better analytics

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Kafka	Event streaming for notification queue	Tech Guide
Redis	Rate limiting and user preference caching	Tech Guide
WebSocket	Real-time in-app notifications	Tech Guide
SignalR	Managed push notification service	Tech Guide

How to Explain It Well

> **Opening**: "A notification service needs reliability and low latency for time-sensitive messages. I'll design for 1B daily notifications across email, SMS, and push."

> **Key talking points**:
>
> - "Priority queues ensure 2FA codes go out before marketing emails"
> - "User preferences checked before sending - respect quiet hours"
> - "Exponential backoff with dead letter queue for failures"
> - "Track delivery status and engagement (opens, clicks)"
> - "Multi-provider setup for failover (SendGrid → SES)"
